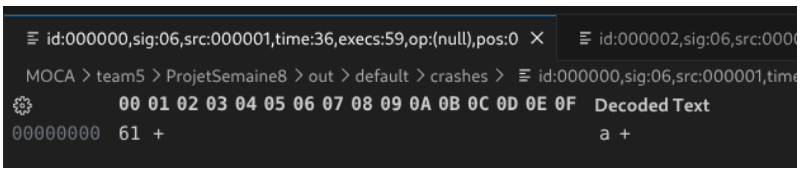
Projet Semaine 8

KALI Axel
KOBZARENKO Polina
SAINT DENIS Leopold
SERVANT Tom

Cette semaine, nous avons analysé et corrigé, en nous appuyant sur les connaissances acquises lors des derniers TPs, les 15 crashes détectés par AFL au cours de la semaine précédente.

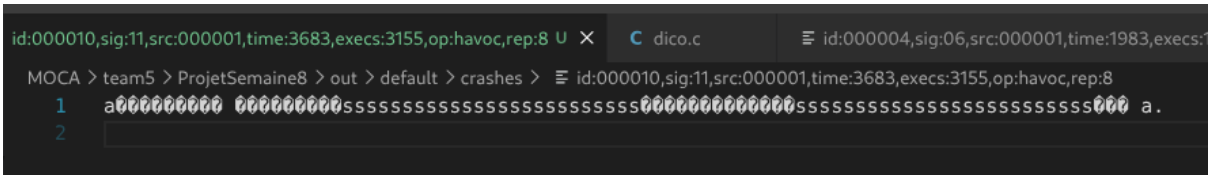
Crashes 0-9, 11, 12 et 14



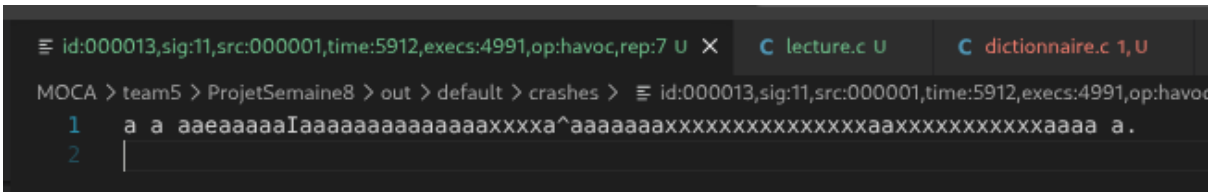
id:000000,sig:06,src:000001,time:36,execs:59,op:(null),pos:0

Analyse : Le test ne se terminait pas par un séparateur, ce qui provoquait un dysfonctionnement dans next_word(), incapable de détecter correctement la fin du dernier mot.
Solution : Nous avons ajouté un traitement spécifique pour gérer ce cas, afin de rendre l'application plus robuste.
Impact : Les crashes id:000001 à id:000009, ainsi que id:000011, id:000012 et id:000014, avaient tous la même cause : l'absence de séparateur final.

Crashe 10 et 13



id:000010,sig:11,src:000001,time:3683,execs:3155,op:havoc,rep:8



id:000013,sig:11,src:000001,time:5912,execs:4991,op:havoc,rep:7

Analyse : À l'aide de gdb, nous avons identifié un dépassement de mémoire dans addToDico(). La fonction tentait de copier un mot plus long que la taille maximale prévue pour lemot dans la structure mot_data_t, dont la taille est fixée à maxSizeWord.

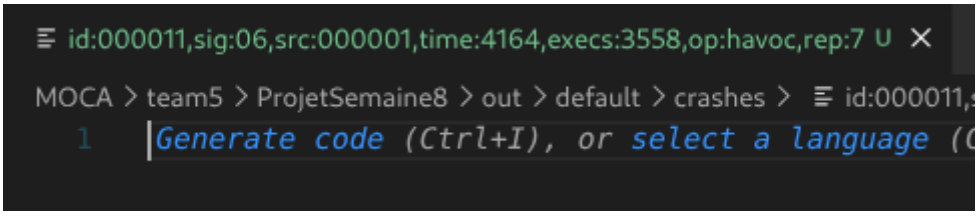
```
poliko@poliko:~/Documents/UGA/S6/MOCA/team5/ProjetSemaine8$ gdb ./dico
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./dico...
(No debugging symbols found in ./dico)
(gdb) run ./tests/test.txt
Starting program: /home/poliko/Documents/UGA/S6/MOCA/team5/ProjetSemaine8/dico ./tests/test.txt
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
malloc(): corrupted top size

Program received signal SIGABRT, Aborted.
__pthread_kill_implementation (threadid=<optimized out>, signo=signo@entry=6, no_tid=no_tid@entry=0) at ./nptl/pthread_kill.c:44
warning: 44     ./nptl/pthread_kill.c: No such file or directory
(gdb) backtrace
#0  __pthread_kill_implementation (threadid=<optimized out>, signo=signo@entry=6, no_tid=no_tid@entry=0) at ./nptl/pthread_kill.c:44
#1  0x00007ffff7d599ff in __pthread_kill_internal (threadid=<optimized out>, signo=6) at ./nptl/pthread_kill.c:89
#2  0x00007ffff7d04cc2 in __GI_raise (sig=sig@entry=6) at ../sysdeps/posix/raise.c:26
#3  0x00007ffff7ced4ac in __GI_abort () at ./stdlib/abort.c:77
#4  0x00007ffff7cee291 in __libc_message_impl (fmt=fmt@entry=0x7ffff7e6e32d "%s\n") at ../sysdeps/posix/libc_fatal.c:134
#5  0x00007ffff7d63465 in malloc_printerr (str=str@entry=0x7ffff7e6c178 "malloc(): corrupted top size") at ./malloc/malloc.c:5832
#6  0x00007ffff7d66f38 in _int_malloc (av=av@entry=0x7ffff7eaaac0 <main_arena>, bytes=24) at ./malloc/malloc.c:4505
#7  0x00007ffff7d67d92 in __GI__libc_malloc (bytes=<optimized out>) at ./malloc/malloc.c:3406
#8  0x0000555555555b15 in insertDico ()
#9  0x0000555555555a61 in insertDico ()
#10 0x0000555555555c15 in addToDico ()
#11 0x000055555555587a in main ()
(gdb)
```

Solution :

a. Limiter la taille des mots acceptés et tronquer ceux qui dépassent.
b. Allouer dynamiquement lemot dans mot_data_t.

Nous avons retenu la solution “b”. Elle évite de restreindre artificiellement la taille des mots du dictionnaire et optimise l’utilisation mémoire, puisqu’on n’alloue que ce qui est nécessaire.

Crashe 11



id:000011,sig:06,src:000001,time:4164,execs:3558,op:havoc,rep:7

Analyse : Le test échouait également en raison de l’absence de séparateur final. Après avoir corrigé ce cas, l’application interprétait désormais un fichier vide comme contenant un mot vide.

Solution : Nous avons ajouté un traitement spécifique pour reconnaître correctement un fichier vide.

Nouvelle erreur détectée : Après cette modification, un **segmentation fault** est apparu. Grâce à **Valgrind**, nous avons identifié la cause : l’application ne gérait pas le cas d’un dictionnaire vide (ce qui peut se produire avec un fichier vide).

Solution : Ajout d’un traitement explicite pour le cas d’un dictionnaire vide.